Metasploit: how to use it with Cerberus

Updated: 2 November 2026 (Cairo)

> **Read this first.** Use Metasploit only on systems you own or have written permission to test. No public website permits Metasploit testing in this project, so the only real test so far was on your own computer (`127.0.0.1`).

1. Choose what you need

Goal	What to install
Run the beginner lab (simulated Metasploit)	Only the project's Python environment. **No Metasploit install needed.**
Run the real Metasploit port scan through Cerberus	The Rapid7 Metasploit Framework (about 1 GB), plus the steps below

The Homebrew Metasploit cask is **disabled**, so `brew install --cask metasploit` will not work. Download the installer from Rapid7 instead.

2. Install the real Metasploit Framework (optional)

Mac (zsh): Download the macOS installer from Rapid7's installation guide and run it. **Not tested on macOS yet.**

Windows (PowerShell): Download the Windows installer from the same guide and run it. **Not tested on Windows yet.**

Check the install afterwards. Open a new terminal and run:

```
msfconsole --version
```


```
msfconsole --version
```


If the command is not found, the program is not on your PATH. Open its installation folder and use the full path to `msfconsole`.

3. Start the RPC daemon (`msfrpcd`) on loopback only

Choose a strong password and keep it private. Use the **same** password in `backend/.env` (step 4).

Mac (zsh):

```
msfrpcd -U msf -P "YOUR_STRONG_PASSWORD" -a 127.0.0.1 -p 55553 -S -n -f
```

Windows (PowerShell): `msfrpcd` is normally in the Metasploit `bin` folder (for example `C:\metasploit-framework\bin`). Check the exact file name in your installation first, then run it with the same arguments:

```
& "C:\metasploit-framework\bin\msfrpcd.bat" -U msf -P "YOUR_STRONG_PASSWORD" -a 127.0.0.1 -p 55553 -S
-n -f
```

What the options mean:

- `-a 127.0.0.1` listens only on your own computer.
- `-p 55553` is the port Cerberus expects by default.
- `-S` turns off TLS. This is only acceptable on loopback.
- `-n` runs without a database. `-f` keeps it in the foreground.

Leave this window open while you use Cerberus. Stop it with `Ctrl+C`. The first console can take about 15–20 seconds to start on a cold daemon.

4. Configure Cerberus (private `.env`)

Edit `backend/.env` in the project. This file is private. Do not commit it. Add or update these lines:

```
MSF_ENABLED=true
MSF_RPC_HOST=127.0.0.1
MSF_RPC_PORT=55553
MSF_RPC_PASSWORD=YOUR_STRONG_PASSWORD
MSF_RPC_SSL=false
MSF_RPC_TIMEOUT=60
```

Restart the backend after changing `.env`. If you set `MSF_RPC_PASSWORD` in your terminal instead, that value overrides the file. Set it for the current window only:

```
$env:MSF_RPC_PASSWORD = "YOUR_STRONG_PASSWORD"
```

```
export MSF_RPC_PASSWORD="YOUR_STRONG_PASSWORD"
```

5. Run the reviewed port scan from Python

Save the code below as `msf_check.py` inside the **backend** folder. It scans only the loopback address and the ports you approve.

```
import asyncio
from app.ai_engine.pentest import ExecutionEngine, PentestContext, ScopeConfig, default_registry

async def main():
    engine = ExecutionEngine(registry=default_registry())
    context = PentestContext(
        target="http://127.0.0.1:80/",
        scope=ScopeConfig(
            allowed_targets=["127.0.0.1"],
            allowed_ports=[22, 80, 443],
            max_impact="proof_of_concept",
        ),
        config={
            "msf_module": "auxiliary/scanner/portscan/tcp",
            "msf_options": {"PORTS": "22,80,443"},
        },
    )
    findings = await engine.execute("metasploit_scanner", context)
    for finding in findings:
        print(finding.severity.value, finding.title, finding.metadata.get("open_ports"))

asyncio.run(main())
```

Run it from the **backend** folder while `msfrpcd` is running:

Mac (zsh):

```
./.venv/bin/python msf_check.py
```

Windows (PowerShell):

```
.\.venv\Scripts\python.exe msf_check.py
```

6. What is allowed and what is refused

Request	Result
`auxiliary/scanner/portscan/tcp` with approved ports	Allowed
Any other auxiliary module	**Refused**
Any exploit module	**Refused**, even at FULL impact
Ports outside the approved list, or more than `PENTEST_MAX_PORTS`	**Refused**
Target outside scope	**Refused** before the daemon is contacted
`testing_window` set in the scope	**Refused** (the window is not enforced yet)

7. Limits to know

- Metasploit is **not** wired into the app's interface, API or AI agent yet.
- Nmap results are not passed to Metasploit automatically.
- The only real Metasploit test was on loopback. Public tests were not allowed.

- Genuine real-daemon tests so far ran on Linux. Mac and Windows installs were not tested.
- Keep `msfrpcd` on `127.0.0.1` with `-S`. Do not expose it to a network.

Next: see **metasploit steps to check up.pdf** to confirm everything works.